

Scikit-learn Documentation

IPL First Innings Score Predictor

1. Project Overview

Project Title

IPL First Innings Score Predictor

Objective

The objective of this project is to predict the final first innings score of an IPL cricket team using Machine Learning and Django. The model estimates the likely total score based on live match conditions such as batting team, bowling team, current score, wickets, overs, and recent run momentum.

This project combines:

- Machine Learning using Scikit-learn
 - Web development using Django
 - Data preprocessing and prediction pipelines
 - User-friendly frontend interface
-

2. Problem Statement

Predicting cricket scores in real-time is useful for:

- Match analysis
- Fantasy sports insights
- Cricket analytics
- Fan engagement

The challenge is to estimate the final innings score accurately using limited live match information.

The project solves this problem by training a Machine Learning regression model on IPL match data.

3. Technologies Used

Technology	Purpose
Python	Core programming language
Scikit-learn	Machine Learning model training
Pandas	Data manipulation and preprocessing
NumPy	Numerical operations
Django	Backend web framework
HTML/CSS	Frontend user interface
SQLite	Database storage
JSON	Model serialization

4. Machine Learning Workflow

Step 1: Data Collection

The IPL dataset was collected containing match information such as:

- Batting team
- Bowling team
- Current runs
- Overs completed
- Wickets lost
- Last 5 overs runs
- Last 5 overs wickets
- Final innings score

Step 2: Data Preprocessing

The dataset was cleaned and transformed before model training.

Preprocessing steps included:

- Handling categorical variables
- Encoding team names
- Feature selection
- Removing invalid or missing data

Step 3: Feature Engineering

Important input features used:

Feature	Description
batting_team	Current batting team
bowling_team	Current bowling team
current_score	Current total score
overs	Overs completed
wickets	Wickets lost
runs_last_5	Runs scored in last 5 overs
wickets_last_5	Wickets lost in last 5 overs

Step 4: Model Training

The Machine Learning model was trained using Scikit-learn.

Algorithm Used

Linear Regression

Why Linear Regression?

- Simple and interpretable
- Suitable for continuous score prediction
- Fast training and prediction
- Works well for numerical estimation problems

5. Scikit-learn Implementation

Libraries Imported

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
```

Train-Test Split

The dataset was divided into:

- Training data: 80%
- Testing data: 20%

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Model Creation

```
model = LinearRegression()
```

Model Training

```
model.fit(X_train, y_train)
```

Prediction

```
predictions = model.predict(X_test)
```

6. Model Evaluation

The model performance was evaluated using:

Mean Absolute Error (MAE)

```
mae = mean_absolute_error(y_test, predictions)
```

Purpose of MAE

MAE calculates the average prediction error between actual and predicted scores.

Lower MAE indicates better model performance.

7. Django Integration

The trained Machine Learning model was integrated into a Django web application.

Workflow

1. User enters match details
2. Django form sends data to backend
3. Scikit-learn model processes input
4. Predicted score is generated
5. Result displayed on webpage

8. Project Architecture

User Input → Django Form → Scikit-learn Model → Prediction → Display Result

9. Files and Modules

File	Purpose
manage.py	Django project management
predictor/views.py	Handles prediction logic
predictor/forms.py	User input forms
predictor/services.py	Prediction services
predictor/ml/train_model.py	Model training script
settings.py	Django settings
home.html	Frontend interface

10. Advantages of the Project

- Real-time score prediction
- Interactive user interface
- Machine Learning integration
- Easy deployment using Django

- Educational use for ML and web development
-

11. Limitations

- Prediction accuracy depends on training data quality
 - Limited to IPL first innings prediction
 - Linear Regression may not capture complex match patterns
-

12. Future Improvements

Possible future enhancements:

- Use advanced algorithms like Random Forest or XGBoost
 - Add live API integration
 - Improve UI/UX
 - Add second innings prediction
 - Deploy on cloud platforms
-

13. Conclusion

The IPL First Innings Score Predictor successfully demonstrates the integration of Machine Learning and web development using Scikit-learn and Django.

The project predicts IPL innings scores using match conditions and provides users with a clean and interactive interface.

This project also highlights the practical implementation of regression algorithms in sports analytics.

14. References

- Scikit-learn Documentation
 - Django Documentation
 - IPL Dataset Sources
 - Python Official Documentation
-

15. Requirements

```
Python  
Django  
Scikit-learn  
Pandas  
NumPy
```

16. How to Run the Project

Step 1

Install dependencies:

```
pip install -r requirements.txt
```

Step 2

Run migrations:

```
python manage.py migrate
```

Step 3

Start server:

```
python manage.py runserver
```

Step 4

Open browser:

```
http://127.0.0.1:8000/
```

17. Output

The web application accepts live IPL match inputs and predicts the estimated first innings total score.

Example:

Current Score	Overs	Wickets	Predicted Score
120	15.0	4	178

18. Author Information

Project: IPL First Innings Score Predictor Technology Stack: Scikit-learn + Django Category: Machine Learning Web Application